🚀 Инструкция по Деплою Balloo Messenger на Сервер

📋 Предварительные Требования

На сервере (31.128.37.165):

- ☑ Ubuntu/Debian Linux
- ☑ Node.js 18+ установлен
- ☑ PM2 установлен глобально
- ☑ Git установлен
- ☑ Nginx установлен (опционально, для reverse proxy)
- ☑ SSL сертификат (Let's Encrypt или другой)

Локально:

- ☑ Git установлен
- ☑ SSH ключи настроены
- ☑ Доступ к серверу по SSH

🔄 Шаг 1: Подготовка Локальной Версии

1.1 Проверка изменений

```
cd ~/Messenger_Balloo_next_ts/messenger

# Проверка статуса
git status

# Проверка diff
git diff
```

1.2 Установка зависимостей

```
npm install
```

1.3 Проверка .env файлов

```
# Проверить .env.production
cat .env.production
```

```
# Убедиться что есть:
# - YANDEX_CLIENT_ID
# - YANDEX_CLIENT_SECRET
# - SMTP_USER=balloo.Messenger@yandex.ru
# - SMTP_PASS=gouewuqhsufqsbgu
# - DATABASE_URL="file:./data/app.db"
```

1.4 Тестовая сборка (локально)

```
NODE_ENV=production npm run build

# Проверить что сборка прошла без ошибок
# Если есть ошибки - исправить перед деплоем
```

🔄 Шаг 2: Загрузка на Git

2.1 Коммит изменений

```
cd ~/Messenger_Balloo_next_ts

# Добавить все изменения
git add .

# Коммит с сообщением
git commit -m "feat: миграция на Better-SQLite3 + email верификация"

# Описание изменений:
# - Миграция с Prisma на Better-SQLite3 (устранение path conflicts)
# - Email верификация с кодом из 7 слов
# - SMTP настроен: balloo.Messenger@yandex.ru
# - Обновлён versions.json (01.06.2026)
# - Автор: Оберюхтин-Кравец Иван Анатольевич
# - Аватарки-восьмиугольники
# - Яндекс авторизация с авто-регистрацией
```

2.2 Пуш на удалённый репозиторий

```
# Проверить remote
git remote -v

# Пуш на main/master
git push origin main
```

```
# Или если ветка master:
git push origin master
```

2.3 Проверка на GitHub/GitLab

```
# Открыть репозиторий в браузере
# Убедиться что все изменения загружены
```

🖥 Шаг 3: Доступ к Серверу

3.1 SSH подключение

```
ssh root@31.128.37.165
# Или:
ssh user@31.128.37.165

# Ввести пароль
```

3.2 Переход в директорию проекта

```
cd ~/Messenger_Balloo_next_ts
```

⚠ ВАЖНО: Миграция с Prisma на Better-SQLite3

Если на сервере уже было запущено приложение с Prisma:

```
cd messenger

# 1. Остановить приложение
pm2 stop messenger-alpha

# 2. Удалить Prisma dependencies
npm uninstall @prisma/client prisma --save-dev

# 3. Установить better-sqlite3
npm install better-sqlite3

# 4. Удалить старую сборку
rm -rf .next

# 5. Инициализировать БД (создаст таблицы если нет)
```

```
mkdir -p data
node scripts/init-db.js

# 6. Проверить таблицы
sqlite3 prisma/dev.db ".tables"

# 7. Пересобрать
NODE_ENV=production npm run build

# 8. Запустить
NODE_ENV=production pm2 start "npx next start -p 3000" --name "messenger-alpha"

# 9. Сохранить
pm2 save

# 10. Проверить логи
pm2 logs messenger-alpha --lines 50
```

Ожидаемые логи:

```
✓ Все таблицы и индексы созданы успешно
✓ База данных инициализирована
```

НЕ должно быть:

```
❌ Invalid `prisma.chat.findUnique()` invocation:
```

📥 Шаг 4: Обновление Кода на Сервере

4.1 Pull изменений

```
git pull origin main
# или:
git pull origin master
```

4.2 Проверка обновлений

```
git log --oneline -5
git status
```

📦 Шаг 5: Установка Зависимостей

5.1 Переход в папку messenger

```
cd messenger
```

5.2 Очистка node_modules (если были проблемы)

```
rm -rf node_modules package-lock.json
```

5.3 Установка зависимостей

```
npm install

# Проверить что nodemailer установлен
npm list nodemailer
# Должно показать: nodemailer@6.9.x
```

💾 Шаг 6: Настройка Базы Данных

6.1 Проверка папки data

```
# Создать папку если нет
mkdir -p data

# Проверить существующую БД
ls -lh data/
# Должно быть: app.db, app.db-wal, app.db-shm
```

6.2 Миграция БД (если нужно добавить новые таблицы)

```
# Если БД старая и нужно добавить новые поля/таблицы:
node -e "
const db = require('./src/lib/database');
console.log('Проверка таблиц...');
const tables = db.prepare(\"SELECT name FROM sqlite_master WHERE
type='table'\").all();
console.log('Таблицы:', tables.map(t => t.name).join(', '));
console.log('✓ Проверка завершена');
"
```

6.3 Проверка таблицы VerificationCode

```
sqlite3 data/app.db "PRAGMA table_info(VerificationCode);"
# Должно показать все поля таблицы

sqlite3 data/app.db "PRAGMA table_info(User);"
# Должно быть поле: emailVerified
```

⚙️ Шаг 7: Настройка Переменных Окружения

7.1 Проверка .env.production

```
cat .env.production
```

7.2 Если нужно обновить SMTP настройки

```
nano .env.production

# Убедиться что есть:
SMTP_HOST=smtp.yandex.ru
SMTP_PORT=587
SMTP_USER=balloo.Messenger@yandex.ru
SMTP_PASS=gouewuqhsufqsbgu
```

7.3 Сохранить и выйти

```
# Ctrl+O, Enter, Ctrl+X в nano
```

🔨 Шаг 8: Сборка Приложение

8.1 Очистка старой сборки

```
rm -rf .next
```

8.2 Production сборка

```
NODE_ENV=production npm run build

# Ожидать завершения (2-5 минут)
# Проверить что нет ошибок
```

8.3 Проверка сборки

```
ls -lh .next/
# Должны быть файлы сборки
```

🚀 Шаг 9: Перезапуск PM2

9.1 Остановка приложения

```
pm2 stop messenger-alpha
```

9.2 Удаление процесса (опционально)

```
pm2 delete messenger-alpha
```

9.3 Запуск нового процесса

```
NODE_ENV=production pm2 start "npx next start -p 3000" --name "messenger-alpha"
```

9.4 Сохранение списка процессов

```
pm2 save
```

9.5 Настройка автозапуска при перезагрузке

```
pm2 startup
# Скопировать команду из вывода и выполнить её
```

📊 Шаг 10: Проверка Работы

10.1 Статус PM2

```
pm2 status
pm2 list
```

10.2 Логи приложения

```
pm2 logs messenger-alpha --lines 50

# Искать:
# ✓ Все таблицы созданы успешно
# ✓ База данных инициализирована
# - Ошибки компиляции
```

10.3 Проверка API

```
# Проверить что сервер отвечает
curl -I http://localhost:3000

# Проверить API версий
curl http://localhost:3000/api/versions

# Проверить регистрацию (тест)
curl -X POST http://localhost:3000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "email": "test-deploy@balloo.su",
    "password": "Test1234!",
    "displayName": "Test Deploy"
  }'
```

10.4 Проверка БД

```
sqlite3 data/app.db "SELECT id, email, displayName, userNumber, points FROM
User ORDER BY userNumber LIMIT 5;"
```

🌐 Шаг 11: Nginx Настройка (если используется)

11.1 Конфиг Nginx

```
nano /etc/nginx/sites-available/balloo.su
```

11.2 Пример конфигурации

```
server {
    listen 80;
    server_name alpha.balloo.su;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

11.3 Активировать конфиг

```
ln -s /etc/nginx/sites-available/balloo.su /etc/nginx/sites-enabled/
nginx -t
nginx -s reload
```

🔒 Шаг 12: SSL Сертификат (Let's Encrypt)

12.1 Установка certbot

```
apt update
apt install certbot python3-certbot-nginx
```

12.2 Получение сертификата

```
certbot --nginx -d alpha.balloo.su
```

12.3 Автоматическое обновление

```
certbot renew --dry-run
```

🔍 Шаг 13: Мониторинг и Логирование

13.1 PM2 мониторинг

```
pm2 monit
```

13.2 Просмотр логов в реальном времени

```
pm2 logs messenger-alpha --lines 100 --nostream
```

13.3 Ротация логов

```
pm2 install pm2-logrotate
pm2 set pm2-logrotate:max_size 10M
pm2 set pm2-logrotate:retain 7
```

🧪 Шаг 14: Финальное Тестирование

14.1 Регистрация нового пользователя

```
curl -X POST https://alpha.balloo.su/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "email": "final-test@balloo.su",
    "password": "Test1234!",
    "displayName": "Final Test"
  }'
```

14.2 Проверка email (консоль или реальный email)

```
# Если SMTP настроен - проверить почту balloo.Messenger@yandex.ru
# Или посмотреть в логах сервера:
pm2 logs messenger-alpha | grep "Verification code"
```

14.3 Проверка через браузер

```
https://alpha.balloo.su
# Зарегистрироваться
# Проверить модальное окно верификации
```

📝 Чек-лист Деплоя

Перед деплоем:

- ☐ Все изменения закоммичены
- ☐ `npm run build` проходит без ошибок локально
- ☐ `.env.production` содержит все переменные
- ☐ SMTP настроен: balloo.Messenger@yandex.ru
- ☐ Git репозиторий обновлён

На сервере:

- ☐ `git pull` выполнен успешно
- ☐ `npm install` завершён без ошибок
- ☐ `npm run build` завершён без ошибок
- ☐ PM2 процесс запущен
- ☐ `pm2 logs` не содержит критических ошибок
- ☐ API отвечает (curl тест)
- ☐ Регистрация работает
- ☐ Email верификация работает

После деплоя:

- ☐ Сайт доступен по HTTPS
- ☐ SSL сертификат валиден
- ☐ Nginx работает корректно
- ☐ PM2 автозапуск настроен
- ☐ Логи ротации настроены
- ☐ Мониторинг работает

🐛 Устранение Проблем

Проблема: PM2 процесс упал

```
# Проверить логи
pm2 logs messenger-alpha --lines 100

# Перезапустить
pm2 restart messenger-alpha

# Если не помогает - удалить и создать заново
```

```
pm2 delete messenger-alpha
NODE_ENV=production pm2 start "npx next start -p 3000" --name "messenger-alpha"
pm2 save
```

Проблема: Ошибки сборки

```
# Очистить и пересобрать
rm -rf .next node_modules
npm install
NODE_ENV=production npm run build
pm2 restart messenger-alpha
```

Проблема: База данных не открывается

```
# Проверить права доступа
chmod 660 data/app.db

# Проверить что процесс имеет доступ
ls -lh data/

# Если нужно - восстановить WAL
sqlite3 data/app.db "PRAGMA journal_mode = WAL;"
```

Проблема: Email не отправляется

```
# Проверить SMTP настройки
cat .env.production | grep SMTP

# Проверить логирование
pm2 logs messenger-alpha | grep -i "email\|smtp"

# В логах будет показан код (если SMTP не настроен):
# [Email] CODE: солнце-месяц-звезда-небо-земля-вода-огонь
```

📞 Контакты и Поддержка

Разработчик: Оберюхтин-Кравец Иван Анатольевич
Email: balloo.Messenger@yandex.ru
Сервер: 31.128.37.165:3000
Домен: alpha.balloo.su

Версия инструкции: 1.0
Дата: 2026-06-01
Статус: ☑ Готово к использованию